# OOP上机实验(4)

**(Version: 0.79 Date: 2026/4/22 Author: wangxp@fudan.edu.cn)**

# 1 实验主题

通过本实验，理解浅复制、深复制和移动，并实现如下目标

- 浅复制、深复制和移动的使用场景；
- 浅复制、深复制和移动的实现；
- 程序代码的功能测试与性能测试。

# 2 实验内容

## 2.0 提供的项目文件

项目提供的文件包括

```
lab4/
├── CMakeLists.txt          # 构建配置
├── README.md               # 项目说明
├── 3rd/
│   ├── benchmark-1.9.5.zip     # benchmark库
│   └── googletest-1.17.0.zip   # googletest库
├── cmake/
│   └── SetupDependencies.cmake # 设置googletest和benchmark库
├── src/
│   ├── bad_buffer.h        # BadBuffer类（已提供，不可修改）
│   ├── buffer_stage1.h     # 阶段1：实现深拷贝的GoodBuffer
│   ├── buffer_stage2.h     # 阶段2：添加移动语义的GoodBuffer
│   ├── test_stage1.cc      # 阶段1测试：BadBuffer问题 + GoodBuffer验证
│   ├── test_stage2.cc      # 阶段2测试：移动语义功能测试
│   ├── bench_compare.cc    # 性能测试（对比阶段1和阶段2）
│   └── CMakeLists.txt      # 构建配置
└── build/                  # 构建目录（不提交）
```

其中需要修改和提交的文件包括

- buffer_stage1.h
- buffer_stage2.h
- test_stage1.cc
- test_stage2.cc
  在修改代码时，不要把原来的标记"// TODO: "删除，要保留标记。

# 2.1 发现问题

从一个"有问题"的 Buffer 类开始

**给定代码：**

```
class BadBuffer {
public:
    BadBuffer(size_t size) : size_(size), data_(new char[size]{}) {}
    ~BadBuffer() { delete[] data_; }
private:
    size_t size_;
    char *data_;
};
```

**要求**:

1、修改buffer_stage1.h文件，在"// TODO:" 后面补写合适的代码；
2、修改test_stage1.cc，在"// TODO:" 后面补写合适的代码；
3、运行运行test_stage1程序，看test_bad_*（三个函数）是否崩溃?
4、运行运行test_stage1程序，看test_good_buffer()是否全通过；

## 2.2 性能优化

**要求**

1、修改buffer_stage2.h文件，在"// TODO:" 后面补写合适的代码；
2、修改test_stage2.cc文件，在"// TODO:" 后面补写合适的代码；
3、运行运行test_stage2程序，确保该程序运行通过；
4、运行bench_compare程序，查看不同运行状态下的性能。

# 3 提交要求

## 3.1 提交代码文件

应该包括下面的这些文件：

- buffer_stage1.h
- buffer_stage2.h
- test_stage1.cc
- test_stage2.cc

## 3.2 提交一份文档

对于bench_compare的运行结果进行分析，**要求的内容格式**

1）粘贴完整的benchmark输出，至少包含：

```
...
BM_Stage1_CopyConstruction/1048576     530881 ns       519031 ns         1445
...
BM_Stage2_MoveConstruction/1048576     332626 ns       329998 ns         2036
...
BM_Stage1_CopyAssignment/1048576       234251 ns       234375 ns         3200
...
```

2）制作对比表格：

| 测试项 | 阶段1时间 | 阶段2时间 | 性能提升 |
|---|---|---|---|
| 构造(1MB) | XXX ns | XXX ns | XX倍 |
| 赋值(1MB) | XXX ns | XXX ns | XX倍 |
| 容器操作 | XXX ns | XXX ns | XX倍 |

3）回答问题。
对于结论：**深复制用时为O(n)，空间开销为O(n)；移动用时为O(n)，空间开销为O(1)**，从这个表格中是否可以说明？为什么？

# 4 关于使用第3方C++库的说明

## 4.1 lab4中使用的第3方库

lab4中使用了googletest和benchmark这两个第3方库。在C++中，对于第3方库的安装、使用，不像python中那般简单，通常会比较复杂。
在本次lab中，提供了一种简单的方法来使用者两个第3方库。如果需要使用其他的库，可以使用类似的方法：前提是第3方库不是很多。

## 4.2 使用第3方库的方式

原则上，使用第3方库都需要把文件下载到本地，然后设置各种路径和选项，构建静态库或动态库；最后在CMakeLists中进行配置。

建议的使用第3方库的方式：

- 把需要的第3方库下载到本地，放在3rd目录下，注意3rd目录是在根目录下；
- 在根目录下创建一个cmake目录，创建一个SetupDependencies.cmake文件；在该文件中创建一个宏(macro，与函数类似)，通过FetchContent处理第三方库。最大的亮点在于使用**FetchContent_Declare**时，通过**url**指定使用的文件是本地文件。大概内容如下

```
macro(setup_dependencies)
    message(STATUS "Setting up local dependencies from 3rd...")
    # --- 1. 禁止 GoogleTest 生成它自己的测试 ---
    # GTest 本身有很多测试，我们只需要库
    FetchContent_Declare(
      googletest
      URL "${CMAKE_CURRENT_SOURCE_DIR}/3rd/googletest-1.17.0.zip"
      DOWNLOAD_EXTRACT_TIMESTAMP TRUE
    )
    set(gtest_force_shared_crt ON CACHE BOOL "" FORCE)
    set(INSTALL_GTEST OFF CACHE BOOL "" FORCE)
    FetchContent_MakeAvailable(googletest)
endmacro()
```

- 如果某个项目需要使用第3方库，那么可以选在在根目录的CMakeLists.txt中，添加下面的两行

```
include(cmake/SetupDependencies.cmake)
setup_dependencies()
```

注意：这两行应该出现在add_subdirectory之前。

# 5 实验要求

## 5.1 实验提交要求

- 如果有源代码提交的要求，请不要改变源代码原来的名字；
- 如果有截屏的要求，把截屏的内容放在PDF文件中(文件名为：学号_实验编号.pdf)，并在前面写明实验内容的编号和源代码名称；
- 把所有要提交的源代码和PDF文档放在同一个文件夹内，把该文件夹压缩；
- 压缩文件支持zip或rar两种形式。

## 5.2 压缩文件命名

压缩文件名为：学号_姓名_实验编号.rar 或学号_姓名_实验编号.zip。例如第1次实验，学号为
22307130022,姓名为张其的学生，提交的压缩文件名为 22307130022_张其_1.rar或22307130022_张其_1.zip）。请大家务必按照命名规则来正确命名文件的名称，否则可能导致评分无法进行，影响实验评分!
请注意：提交的文件不包括*.obj和*.exe，只需要包含*.cpp 和*.h 文件即可。如果有说明，请把说明包含在文本文件中，例如代码demo.cpp的说明文件名为demo.cpp.txt。

## 5.3 截至时间

截至时间通常为本周五晚上12点前，请大家安排时间准时完成作业。